



Final Report: Using Machine Learning to Predict Traffic Volume

Higher Diploma in Science in Computing

by: **Seán Carley**

September 2026

Word Count: **3932/4000 words** ¹

¹Excluding table of contents, headers, references, captions, bibliography, listings

Contents

1	Introduction	6
2	Review of Existing Product: PTV Optima	7
3	Project Scope and Objectives	9
3.1	Project Objective	9
3.2	Project Scope	9
4	Implementation	10
4.1	Current Status	10
4.2	Project Elements	11
4.2.1	Data Gathering	11
4.2.2	Relational Database	19
4.2.3	Machine Learning	19
5	Machine Learning	27
5.1	Linear Regression	27
5.2	K-Nearest Neighbour	28
5.3	Support Vector Machine	29
5.4	Decision Tree	29
5.5	Time Series	30
6	Conclusion	31
6.1	Future Development	31

7	References/Bibliography	32
8	Github Repository	33

List of Figures

1	manually entering report link to browser	13
2	HTML returned by <code>print(soup.prettify())</code>	13
3	Database ERD	19
4	Regression Line using the “epoch” feature	27
5	Regression Line using the “epoch” feature	29
6	Decision Tree	29

Listings

1	BeautifulSoup example	13
2	creating webdriver object	14
3	action chain	15
4	web driver wait - 1	16
5	web driver wait - 2	16
6	ProcessReport __init__ Method	17
7	ProcessReport make_soup Method - 1	17
8	ProcessReport make_soup Method - 2	18
9	ProcessReport make_soup Method - 3	18
10	ProcessReport make_soup Method - 4	18
11	GetTrainingData - 1	21
12	GetTrainingData - 2	22
13	GetTrainingData - 3	22
14	MachineLearning - __init__	23
15	MachineLearning - get_test_training_dataset	24
16	LinearReg - __init__	25
17	KNearestNeighbour - __init__	25
18	SupportVectorMachine - __init__	25
19	Decision Tree - __init__	25
20	Decision Tree - __init__	25

1 Introduction

Traffic Congestion is a significant problem that affects most urban areas. In Dublin the M50 motorway, a ring road starting near Dublin Airport to the north of the city and running to the M11 near Shankill, to the south, is Ireland's busiest motorway. With connections to most of the major national routes, it plays a major role in the distribution of goods and services to the island of Ireland.

In a recent article on RTE.ie, Transport Infrastructure Ireland (TII), the state agency responsible for managing and developing the public road and transport infrastructure in Ireland, are quoted saying:

...we're at capacity. We can't make it any wider, and you can't make it a double decker...[1]

This does not bode well for road users who are already saying that journeys that used to take under 1 hour, are now exceeding that significantly. There are numerous proposals to try help improve the situation (i.e. a proposal to build an outer ring-road); however, these take a considerable amount of time to plan and design. And even then, approval is not guaranteed. This process could take decades to be completed, before the actual building starts. This means that the current infrastructure will have to cope with the ever increasing demand.

Can machine learning technology be used to help improve the efficiency of the current infrastructure?

2 Review of Existing Product: PTV Optima

“Real-time traffic management software for multimodal networks” [2]

PTV Optima is a traffic management software that connects multiple data sources to help operators identify solutions to manage congestion, road closures, etc... and simulate the resulting network performance

“Expanding transportation infrastructure is costly and might not result in less traffic bottlenecks. While congestion is hard to predict, our traffic management software PTV Optima unlocks your city’s true potential. With reliable forecasts, high-quality real-time data and enhanced decision support, you will deploy the best strategy for your network” [2]

Some of the functions of the PTV Optima System:

- Traffic Data analysis
- **Traffic Forecast**
 - combining Machine Learning and Transportation Models to assist in managing both planned and unplanned events
- Customised Monitoring
- Traffic Scenarios
- Automation
- Adaptive Signal Control
- Traffic Micro-Simulation
- Public Transport Operations
- Journey Planner
- Integrated Traffic Management System

PTV Optima has been deployed in multiple location throughout Europe and Aisa.

In Erfurt Germany, a historic city, with narrow streets, the option to expand the size of the streets is not an option. Therefore, in order to accommodate the increasing population, and housing density, a Traffic Management system is needed. The Optima system runs on live data such as traffic levels (from detectors), current parking situation, information from the public transport network, and traffic restrictions (e.g. road works, diversions, etc...) and provides continuously updating solutions to the current traffic situation. An example of a situation where the system assists would be controlling traffic in the city centre. Traffic light sequences can be modified so that more vehicles are allow to leave the city centre than are allowed to enter. ²

In Piedmont Italy, The Optima system uses a transport model, from another PTV product, which shows a “typical” day. This is combined with real-time data to adjust the volume, speed, and capacity in the model to predict the impact of unforeseen incidents. The Optima system can provide a 60 minute forecast which is recalculated every 5 minutes. Providing a rolling view of the traffic. With Piedmont being close to the Alps, the nearby tunnels can be closed due to adverse weather conditions. This could cause unusual congestion patterns. The Optima system can recognise the change and suggest a strategy to the traffic managers. ³

²https://www.ptvgroup.com/sites/default/files/202509/PTV-Optima_SUC_Erfurt_EN_2017.pdf

³https://www.ptvgroup.com/en/ptv_optima___success_storie___piedmont?inline

3 Project Scope and Objectives

3.1 Project Objective

The main objective of this project will be to examine if Machine Learning can be used to accurately predict future traffic volumes. What algorithms can be used, and which ones will give the best prediction. Can these be used to help authorities manage traffic flow, help haulers plan deliveries to avoid periods of high congestion, the private motorist plan the all important journey?

3.2 Project Scope

The first requirement of this project will be to gather data. In the real world, the data would be gathered using monitoring equipment on the road (i.e. cameras, induction loops, pneumatic counters, etc..). However, for this project, the plan is to use data from TII's <https://trafficdata.tii.ie> [3] website. This will require the use of web scraping to retrieve the data and the development of a database to store it.

It is intended that the Python library BeautifulSoup will be used for this, however, as the official documentation has not been updated in some years, another similar library (i.e. Selenium or Scrappy) may need to be used. A MySQL database will be used to store the gathered data. MySQL was chosen as it is readily available and has a supporting Python library.

The next requirement will be to explore what machine learning algorithms can be used. This will be achieved using the Pandas, Numpy, Matplotlib, SciKit-Learn python libraries. These Libraries provide various methods to structure the data, methods for machine learning algorithms and visualisation.

4 Implementation

4.1 Current Status

Project timeline:

Milestone	Date Due	Week
Proposal Submission	06/07/2026	0
Interim Submission	21/08/2026	6
Final Report Submission	25/09/2026	12
Artefact Submission	28/09/2026	12

The initial project plan outlined in the Project Proposal document was as below:

12 weeks from Project Proposal Submission to Artefact Submission

- **Weeks 1 - 2:** Work to be carried out on data gathering and database design.
- **Weeks 3 - 4:** Work to be carried out on machine learning element. Create queries to retrieve data and structure it to be usable. Run some simple machine learning algorithms on the data and look at generating some initial graphs/visuals.
- **Weeks 5 - 6:** Review existing work and prepare interim submission and demo for the end of week 6
- **Weeks 7 - 8:** Expand machine learning element and finalise which algorithms will be included in final submission.
- **Weeks 9 - 10:** Develop a dashboard to display information to user based on the criteria entered.
- **Weeks 11 - 12:** Review work and prepare final report and final demo.

The difficulties experienced in the data gathering stage of this project did have an impact on the overall quality of the final product. The machine learning element needed a lot more time to complete. The coding was relatively easy, however it was investigating how to evaluate the performance of the models that needed a lot more time. One take away is to not just focus on the coding element, but what is needed to accompany it. This is the main lesson learnt from this project

4.2 Project Elements

4.2.1 Data Gathering

In the “real world” the vehicle count would be obtained directly from the road side. There are a number of methods that could be used for collecting this information, for example pneumatic counters, cameras, induction loops.

There are pros and cons to each technology:

Pneumatic Counter

- + easy to install
- easily damaged/vandalised
- accuracy of classification vehicles

Camera

- + easy to install
- + can classify vehicles
- expensive
- potential privacy implications

Induction Loop

- + can classify vehicles
- + more robust
- expensive
- disruption if installing on existing road

Traffic counters record hourly traffic volumes by vehicle class, e.g. motorcycle, car, goods vehicles distinguished by number of axles etc... with up to twelve vehicle classes being identified. Vehicles are detected by passing over loops embedded in the road surface. The data enables identification of vehicle profiles and classes. [4]

Ideally the count data would be stored on a database and an API (Application Programming Interface) would be used to retrieve the data. This would improve performance and security. For this project, however, there was no API to retrieve the data. So an alternate approach was used.

The initial approach was to use Beautiful Soup to perform web scraping from the TII Traffic Website <https://trafficdata.tii.ie> [3] website. However, there were some difficulties experienced when I tried to use the library to access the various reports on the TII website.

The initial approach was to use Beautiful Soup with the URL that would navigated directly to the required report. The URL is structured as below:

https://trafficdata.tii.ie/tfdaysreport.asp?sgid=abc1efghJklMNopqr_trSB&spid=A1B2C3D4E5F67890&reportdate=2026-08-18&enddate=2026-08-18&dim1bin=1&intval=2

- protocol & domain: <https://trafficdata.tii.ie/>
- report type: tfdaysreport.asp?
- additional parameters:
 - session id: &spid=A1B2C3D4E5F67890 (12 digit hexadecimal no.)
 - start date: &reportdate=2026-01-01 (YYYY-MM-DD format)
 - end date: &enddate=2026-01-02 (YYYY-MM-DD format)
 - vehicle class: &dim1bin=1 (number determines if all vehicles or individual classes are shown)
 - direction: &dir=1 (number determines if all directions or individual lanes are shown)
 - interval: &intval=2 (number determines the granularity of the report (5 mins - 24hrs))

The plan was to change to parameters to retrieve the required report results, for example change the “reportdate” and “enddate” to the required date range, set the “intval” to 5 minute increments, set the direction to show each lane in both directions. Manually opening a browser and pasting the modified link in to the address bar returns the expected page and data:

Using the *Requests* Python library the html returned was not the expected report with the traffic counts. To try get the desired response, the headers from a valid request sent via a browser were copied and these were included in the requests.get call. This did not resolve the issues. Upon further investigation, the leading theory is that the requests call doesn’t wait for any JavaScript to run on the page and returns what ever is loaded initially. The html captured is what was initially provided by the server, as shown by the second image in **Figure 2**, the rendered html showing the “loading” page.

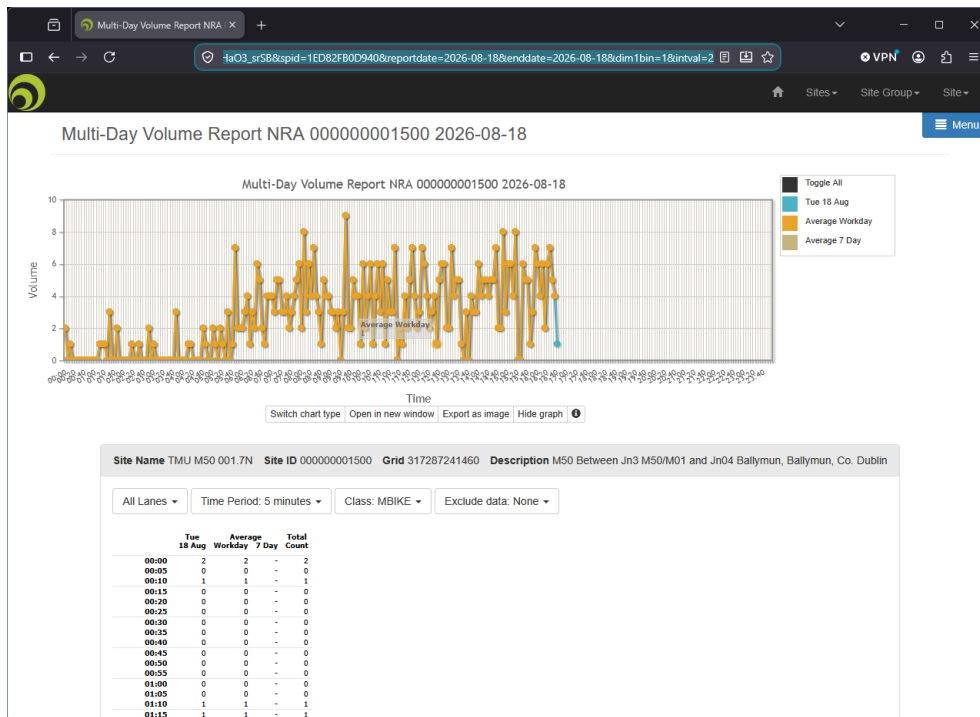


Figure 1: manually entering report link to browser

```

1 >>> from bs4 import BeautifulSoup
2 >>> import requests
3 >>> page = requests.get("** URL omitted for readability **")
4 >>> print(page.status_code)
5 200
6 >>> soup = BeautifulSoup(page.content, 'html.parser')
7 >>> print(soup.prettify())

```

Listing 1: BeautifulSoup example

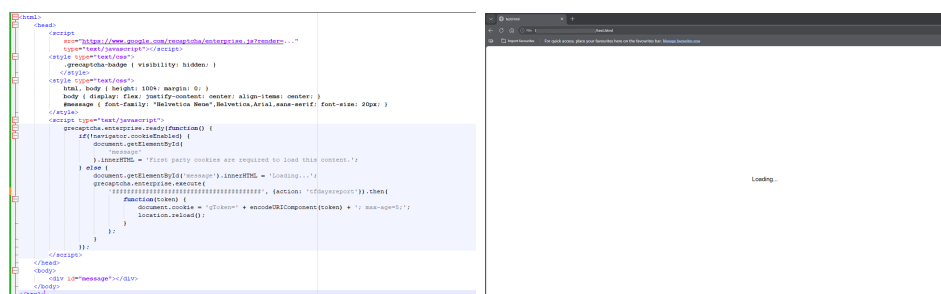


Figure 2: HTML returned by print(soup.prettify())

Another complication relating the the requests call, is that there is no browser

used, so each new call is like opening the page in an Incognito (chrome) session. As a result, none of the session IDs are the same and therefore cannot be used from one session to another.

What was needed was a way to simulate a “real” user using a browser, accepting cookies, simulating mouse movements, entering text to search fields, and clicking on buttons/links.

Selenium is able to do this, and it has a python library.

Selenium automates browsers. That's it! What you do with that power is entirely up to you.[5]

Using the below sites: <https://www.selenium.dev/documentation/> <https://www.scrapingbee.com/blog/python/>

and Google and Stackoverflow, I was able to create a program to return the html from the TII website using Selenium.

To use Selenium, a web driver object is required. This is created in the following way:

```
1 from selenium import webdriver
2 from selenium.webdriver import ActionChains
3 from selenium.webdriver.chrome.options import Options
4 from selenium.webdriver.common.by import By
5 from selenium.webdriver.support import expected_conditions as
   EC
6 from selenium.webdriver.support.ui import WebDriverWait
7
8 options = Options()
9
10 options.add_argument('--headless=new')
11 options.add_argument('--window-size=1920,1080')
12
13 driver = webdriver.Chrome(options=options)
14
15 driver.get(url)
16
17 action = ActionChains(driver)
18
19 wdw = WebDriverWait(driver, 10)
```

Listing 2: creating webdriver object

The “options” variable is a list of options to be used in creating the driver object. For this project the following were used:

- “-headless=new” lets Selenium run the session without showing the browser. If this option is omitted a browser will be shown on the screen ⁴
- “-window-size=1920,1080” resizes the resolution of the browser. if this option is omitted, the default screen size of 800x600 will be used. ⁵

There are various other options which were not needed for this project.

`driver = webdriver.Chrome(options=options)` initiates the driver object to use the Chrome browser driver and apply the required options. Other browser drivers are available, i.e. Firefox, Edge, etc... and depending on which browser is used the appropriate driver would need to be imported.

`driver.get(url)` starts the specified browser driver and navigates to the provided url.

`action = ActionChains(driver)` assigns the ActionChains method to the variable which allows the chaining of browser interactions for example:

```
1 action.move_to_element(search).click(search).send_keys("TMU
   M50 001.7N").perform()
```

Listing 3: action chain

The above sets up a chain of actions to be performed:

- move to the web element assigned to the variable “search”
- click on the element
- enter the provided characters

The perform keyword at the end instructs the program to perform the chain. This allows Selenium to mimic user behaviour.

One of the biggest challenges in automating browser interactions is knowing when a web element has been loaded. If an interaction was triggered on an element that was not loaded, the process would come to a stop. Waits could be added to slow the program down, but how long should you wait?

too long of a wait, the program runs slowly

⁴currently the program does not run with “-headless=new” enabled

⁵responsive elements may not display correctly at this resolution

too short if a wait, the target element is not available to be interacted with

This is where WebDriverWait comes in:

```
1 WebDriverWait(driver, 10).until(  
2     EC.presence_of_element_located((By.XPATH, '//*[@id="class  
-tab"]'))  
3 )
```

Listing 4: web driver wait - 1

WebDriverWait takes 2 arguments, the driver object, and an integer value. This integer value is a timeout, in seconds, ensuring that the program doesn't hang. Combining this with the ".until" and "EC.presence_of_element..." methods, allows time for the required web element to load before any action is performed.

This is also how the web element was assigned to the variable in the action chain example:

```
1 search_result = wdw.until(  
2     EC.presence_of_element_located(  
3         (By.XPATH, '//*[@id="mapSearchResults"]'))  
4 )
```

Listing 5: web driver wait - 2

The above waits until the expected condition (EC) of the element with an id of "mapSearchResults" is located, using its xpath in this case. If it is not found within the specified timeout, an exception will be thrown, which will have to be managed.

Once the required page with the report for the specified dates had been obtained from the source (<https://trafficdata.tii.ie> [3]), the required data had to be extracted. BeautifulSoup was used to do this.

Beautiful Soup is a library that makes it easy to scrape information from web pages. It sits atop an HTML or XML parser, providing Pythonic idioms for iterating, searching, and modifying the parse tree[6]

A class called ProcessReport was created to handle BeautifulSoup. It requires the report object which contains the html from the selenium process explained above.

The __init__ method establishes some instance variables (i.e. variables belonging to a specific object).

The make_soup method is where the data is processed.

```

1 def make_soup(self):
2     '''
3         take the html from get_report.py to prepare it to be
4         added to the
5         database
6     '''
7     soup = BeautifulSoup(self.report, 'html.parser') #
    create a BeautifulSoup object
    events = []

```

Listing 6: ProcessReport __init__ Method

The report object is passed to BeautifulSoup along with the type of parser to be used (in this case html). An empty list is also created which will contain the data to be stored in the database at the end.

In order to access the required data from the report object, the structure of the html page needs to be known so that BeautifulSoup knows where to look for it. In the below the for loop is looking for the html “tbody” tag that is immediately followed by the “tr” body tag:

```

1 for tr in soup.select('tbody tr'):
2     for td in tr.select('td'):
3         # the td element that has a rowspan of 16 has the
4         date
5         # the below if statement checks if the current td
6         element and
7         # if it has a rowspan of 16 it retrieves the data
8         if (td.has_attr('rowspan')) and (
9             int(str(td).split('"')[1]) == 16):
10            td_str = str(td).replace(
11                '<td rowspan="16">', '').replace('</td>', '')
12            date_str = td_str.split(' ')
13            year = int(date_str[3])
14            month = self.months[date_str[2].lower()]
15            day = int(date_str[1])
16            dow = self.days[date_str[0].lower()]

```

Listing 7: ProcessReport make_soup Method - 1

An inner for loop then looks for html “td” tags and extracts the data. If the “td” element has a “rowspan” attribute with the value of 16, it contains the date information. The “if” statement checks for this and assigns the year, month, day and day of the week (dow) to the corresponding variables.

“td” elements that have “class” and “data-a” attributes contain the the count for the specified time range and vehicle class. The below section of the make_soup method extracts this data and assigns it to the corresponding variables:

```

1 if (td.has_attr('class')) and (td.has_attr('data-a')):
2     td_str = str(td).split('')
3     if td_str[1] == '':
4         # week days with no event have a class that is
5         # interpreted as an empty string, so to avoid having
6         # empty string as a key in the dict, just assign int
7         # to the list
8         event_class = 0
9     else:
10        # check the events dict for the int value to
11        # represent
12        # the event, change to lowercase to match the key
13        event_class = self.events[td_str[1].lower()]
14        data_a = td_str[3].split(',') # split the data-a attr
15        direction = int(data_a[0])
16        hour = int(data_a[2])
17        vehicle_class = int(data_a[3])
18        # extract the vehicle count for the particular hour
19        raw_count = td_str[4].replace('>', '').replace('</td', '')
20    vehicle_count = 0

```

Listing 8: ProcessReport make_soup Method - 2

The below try - except is used to catch errors throw as a result of no or incompatable data being available for the count. If an error is thrown, the count value is left at 0 as initialised above, and the application can continue. Otherwise the count value is assigned to the vehicle_count variable.

```

1 try:
2     vehicle_count = int(raw_count)
3 except ValueError as ve:
4     print(f'{ve} returned for the vehicle class {
5         vehicle_class{day}/{month}/{year} {hour}:00')
6     print('The count for this has been set to 0')

```

Listing 9: ProcessReport make_soup Method - 3

Once all the variables are set, they are added to a list and this list is added to the events list created earlier. The events list is returned so that it's contents can be added to the database.

```

1 # add the processed data to the events list to be used to
2 # add the data to the database
3 events.append([year, month, day, dow, hour, event_class,
4               direction, vehicle_class, vehicle_count])
5

```

```
6 return events
```

Listing 10: ProcessReport make_soup Method - 4

4.2.2 Relational Database

The current design for the Relational Database is as below:

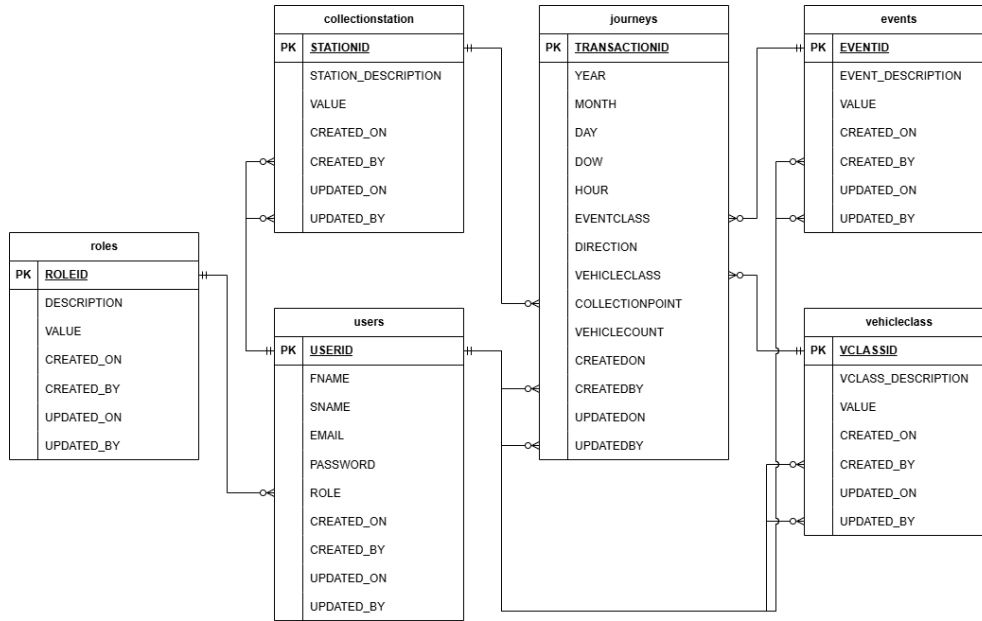


Figure 3: Database ERD

The information from the roadside will be stored in the “journeys” table. All of the information will be stored as integers to allow for faster processing. The data in the majority of the columns will be a Foreign Key to the corresponding table which will contain further information. This information will be mainly used for the Dashboard, so that meaningful information can be provided to the user.

4.2.3 Machine Learning

For the Machine Learning section of the application, the intention was to evaluate a number of algorithms to compare their suitability.

In machine learning, there are 2 prediction problems, Classification and Regression. Classification tries to predict the class label of while regressions tries to

predict a continuous value. For this project, as the objective is to try predict the expected volume of traffic for a given time period, regression algorithms will be used. The below Machine Learning Algorithms were evaluated:

- Linear Regression
- K-Nearest Neighbour (KNN)
- Support Vector Machine (SVM)
- Decision Tree Regressor
- Time Series

To implement the the algorithms in Python the Pandas, Numpy, Sci-Kit Learn library were used.

“pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.” [7]

“NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.” [8]

“Scikit-learn is an open source machine learning library that supports supervised and unsupervised learning. It also provides various tools for model fitting, data preprocessing, model selection, model evaluation, and many other utilities.” [9]

Before starting to train any models or make a prediction, the data to be used needed to be retrieved from the database and processed in to usable features. The table on the database has the following columns:

- Year: the year taken from the date
- Month: the month taken from the date
- Day: the day of the month taken from the date

- DOW: the day of the week (i.e. “Monday”, “Tuesday”, etc...)
- Hour: the hour of the day the count applies to
- Event: if there was a special event that might impact the volume (i.e. weather, accident, etc...)
- Direction: the direction of the traffic (i.e. North, South)
- Vehicle Class: the vehicle class descriptor (i.e. “Car”, “Motorbike”, etc...)
- Count: the number of vehicles counted for the vehicle class on the date and time

The above columns are used as the features for the models. Using all the features does not guarantee an accurate, efficient model, and as a result some experimentation on which features to use needed to be carried out. For example, using the year, month and day was expected to be a good baseline, however 1st January 2023 was a Sunday, while in 2024 it was a Monday, and in 2025 it was a Wednesday. There is a difference between the daily volume on a Monday compared to a Wednesday and even more so for a weekend day. This is one reason to include the DOW as a feature.

“GetTrainingData” class was created to retrieve the data from the database and put it in to a numpy array, with the “get_db_data” method creating the connection to the database and creating the numpy array:

```

1 def get_db_data(self):
2     ''' connect to the data base a retrieve the data to be
3     used '''
4     # SQLAlchemy
5     # mysql+mysqlconnector://<user>:<password>@<host>[:<port>]
6     # /<dbname>
7     engine_url = f"mysql+mysqlconnector://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@{os.getenv('DB_HOST')}{int(os.getenv('DB_PORT'))}/{os.getenv('DB_NAME')}"
8     engine = create_engine(engine_url, poolclass=NullPool)
9     query_columns = 'COUNT_YEAR, COUNT_MONTH, COUNT_DAY, COUNT_DOW, COUNT_HOUR, COUNT_EVENT_CLASS, COUNT_DIRECTION, COUNT_VEHICLE_CLASS, COUNT'
10    with engine.begin() as db_conn:
11        df = pd.read_sql_query(sa.text(
12            f'select {query_columns} from counts where COUNT_HOUR between 6 and 19 order by COUNT_YEAR, COUNT_MONTH, COUNT_DAY, COUNT_HOUR;'
13        ), db_conn)
14        db_conn.close()

```

```
13 self.pre_proc_data = np.array(df)
```

Listing 11: GetTrainingData - 1

The data was then processed to format the date to “YYYY-MM-DD”, to format the time to “HH:MM” format and direction to create a key to be used in a python dictionary.

```
1 y = self.num_str(a[0])
2 m = self.num_str(a[1])
3 d = self.num_str(a[2])
4 # set data in required format (YYYY-MM-DD) for use in dict
  keys
5 str_date = f'{y}-{m}-{d}'
6 # set h variable for use in dict keys
7 h = self.num_str(a[4]) + ':0'
8 # set epoch value, this allows the time values to be combined
  into
9 # one value
10 epoch = int(dt.datetime(a[0], a[1], a[2], a[4], 0).timestamp())
11 # set the dir variable to be used in the dict keys
12 if a[6] == 0:
13     dir = 'north'
14 if a[6] == 4:
15     dir = 'sout'
16 # set the week of the year (wy) variable (possible values:
  0-53)
17 wy = int(dt.datetime.strptime(str_date, '%Y-%m-%d').strftime(
  '%W'))
18 # set the key to be used in the dicts
19 str_key = f'{str_date} {h} {dir}'
```

Listing 12: GetTrainingData - 2

The dictionaries for each vehicle class were further processed to sum the counts of each for each time period.

```
1 for dir in self.dirs:
2     for k, v in dir.items():
3         # sum the totals for all vehicle classes and add to
4         # dict_totals dict dropping the vehicle class
5         if k not in self.dict_totals:
6             self.dict_totals[k] = [v[0], v[1], v[2], v[3], v
7             [4], v[5],
8             v[6], v[7], v[9], v[10]]
9         else:
10            self.dict_totals[k][8] = self.dict_totals[k][8] +
            v[9]
11 # create an empty list
```

```

11 lst = []
12 # populate the empty list which will be used to create the
    numpy array
13
14 for k, v in self.dict_totals.items():
15     lst.append(v)
16 # create the numpy array
17 self.ml_arr = np.array(lst)

```

Listing 13: GetTrainingData - 3

Using Sci-kit Learn, a lot of the methods required for each algorithm were the same and as a result it was possible to break them up in to classes, whereby the common methods were include in a "Parent" class (MachineLearning.py), and the more specific methods were separated in to dedicated classes.

The "MachineLearning" class contains the common methods required by the MachineLearning Algorithms. The `__init__` method establishes the instance variables. The columns variable passed to the method lists the required features, and the ml_array passed is the numpy array created from the GetTrainingData class.

```

1 class MachineLearning:
2     ''' parent class for machine learning algorithms '''
3
4     def __init__(self, ml_array, columns):
5         self.ml_array = ml_array
6         self.columns = ['year', 'month', 'day', 'dow', 'woy',
7                         'hour', 'event', 'direction', 'count', 'epoch']
8         self.drop_columns = []
9         self.required_columns = columns
10        self.df = None
11        self.X = None
12        self.y = None
13        self.X_train = None
14        self.X_test = None
15        self.y_test = None
16        self.model = None
17        self.prediction = None
18        self.year = None
19        self.month = None
20        self.day = None
21        self.hour = None
22        self.epoch = None

```

Listing 14: MachineLearning - `__init__`

The "MachineLearning" class also has the "get_test_training_dataset" method. This method takes the numpy array provided when instantiating the class and

turns it into a pandas dataframe. The pandas data frame can be manipulated using included methods to keep, use or modify the data. This method also splits the data in to two data sets. set X contains the required features and set y contains the target, i.e. the count.

```
1 def get_test_training_dataset(self):
2     # create dataframe from ml_array
3     self.df = pd.DataFrame(self.ml_array, columns=self.
4                             columns)
5
6     # drop the columns that are not being used
7     for column in self.columns:
8         if column not in self.required_columns:
9             self.drop_columns.append(column)
10
11     #create the X and y datasets
12     self.X = self.df.drop(self.drop_columns, axis=1)
13     self.y = self.df['count']
14
15     # create the train and test datasets
16     self.X_train, self.X_test, self.y_train, self.y_test =
17     train_test_split(
18         self.X, self.y, test_size=0.4, random_state=79)
```

Listing 15: MachineLearning - get_test_training_dataset

The “train_test_split” method, from Sci-kit Learn, is used to further split the X and y datasets into train and test datasets. The “test_size” attribute tells how much of the data is to be used for testing (0.4 allocates 40% of the data for testing). The “random_state” attribute provides repeatability of the model.

The Machine Learning class also contains the “make_prediction” method. It processes the request omitting the features that were not included in the training of the model and returns the predicted data frame.

Separate classes were created for each of the Machine Learning algorithms used:

- LinearReg.py: Linear Regression
- KNearestNeighbour.py: K-Nearest Neighbour
- SupportVectorMachine.py: Support Vector Machine
- DecisionTree.py: Decision Tree
- TimeSeries.py: Time Series

All the classes inherit from the parent class MachineLearning:

```
1 class LinearReg(MachineLearning):
2     ''' run Linear Regression machine learning algorithm '''
3     def __init__(self, ml_array, columns=['year', 'month', '
    day'],
4         ml_type='Linear Regression'):
5         super().__init__(ml_array, columns)
6         self.ml_type = ml_type
```

Listing 16: LinearReg - __init__

```
1 class KNearestNeighbour(MachineLearning):
2     ''' run KNN machine learning algorithm '''
3     def __init__(self, ml_array, columns=['year', 'month', '
    day'],
4         neighbors=42, ml_type='K Nearest Neighbors (KNN)':
5         super().__init__(ml_array, columns)
6         self.neighbors = neighbors
7         self.ml_type = ml_type
```

Listing 17: KNearestNeighbour - __init__

```
1 class SupportVectorMachine(MachineLearning):
2     ''' run Support Vector Machine '''
3     def __init__(self, ml_array, columns=['year', 'month', '
    day'],
4         ml_type='Support Vector Machine'):
5         super().__init__(ml_array, columns)
6         self.ml_type = ml_type
```

Listing 18: SupportVectorMachine - __init__

```
1 class DecisionTree(MachineLearning):
2     ''' run Decision Tree machine learning algorithm '''
3     def __init__(self, ml_array, columns=['epoch'],
4         ml_type='Decision Tree'):
5         super().__init__(ml_array, columns)
6         self.ml_type = ml_type
```

Listing 19: Decision Tree - __init__

```
1 class TimeSeries(MachineLearning):
2     ''' run Time Series machine learning algorithm '''
3     def __init__(self, ml_array, columns=['year', 'month', '
    day'],
4         ml_type='Time Series Analysis'):
5         super().__init__(ml_array, columns)
```

Listing 20: Decision Tree - __init__

Some of the algorithms require additional attributes, for example K-Nearest Neighbours needs the K value to instruct how many neighbours to use in the prediction. The methods that take these additional attributes are redefined in the class so that the method from the parent class is not used.

5 Machine Learning

Machine learning allows computers to process data and “learn” for it by identifying patterns. These patterns can be used to make predictions. Depending on the data and the prediction required there are 2 possible ways to predict. Classification and Regression. Classification uses predefined features and makes a prediction based on what features are provided for the prediction. Regression uses the features and a numerical value, and tries to identify the pattern. For this project the goal is to predict traffic volume for a given future date/time. As a result, algorithms that are suited to regression problems will be used.

5.1 Linear Regression

Linear regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables. [10]

It tries to fit a straight line to the data, and uses this line to make predictions. For this project the features (independent variables) are entered and the model tries to make a prediction using the line.

Below is the Regression Line using the “epoch” feature⁶. Using just one feature simplifies the graph.

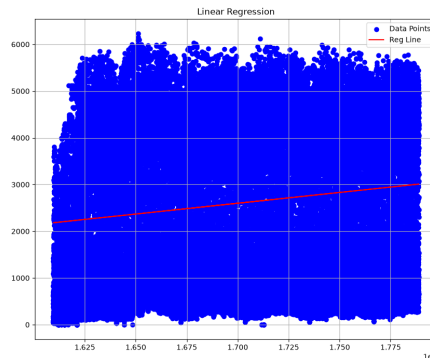


Figure 4: Regression Line using the “epoch” feature

The above shows that the regression line does not fit the data well

⁶the epoch is generated from the DateTime Python library. It is the count of seconds since 01/01/1970

The below table shows some of the stats for the Linear Regression Model using various combinations of features and the predicted value for 09:00 on 01/09/2029⁷:

Feature	MAE	RMSE	Epln Var	Value
all features	844.4	1087.9	0.0737	4316
year, month, day	838.7	1078.7	0.1452	4374
year, dow, woy	817.6	1042.2	0.0843	4664
epoch	844.4	1084.9	0.0737	4316

The Root Mean Square Error (RMSE) is quite high. The units of the RMSE are the same as the target variable. So taking the actual figure for 01/09/2026 at 09:00 being 4455 the RMSE of 1084 is almost 25%. This is a significant error. What is confusing is that the predicted figure is quite close to the actual figure. Is this a coincidence? Running a prediction for 05/09/2026 at 20:00, a Saturday, the value returned is 3008, while the actual count was 2222. a difference of 786. The conclusion reached for this is that for “rush hour” times (07:00 - 18:00) counts are in the 4,000’s, while outside of this time, the counts range from 300 - 2500, which leads to a lot of out liers and affects the regression line.

5.2 K-Nearest Neighbour

The below graph shows the each hour period from 01/01/2021 - 31/08/2026. The features used were “year”, “month”, and “day”. The hue has been applied to the hour. The result is that between 20:00 - 08:00 , the traffic count is below 2,500 for the most part, and for 08:00 - 20:00 the traffic count is above 2,500. This means that using a low number of neighbours, the prediction is not going to vary a lot, leading to the high RMSE value as shown on the below table. Using a high number of neighbours impacts on the performance of the model. On the computer used K exceeding 250 caused the application to stall, and no results were returned.

Feature (k)	MAE	RMSE	Epln Var	Value
all features (90)	632.44	776.3	0.7979	2621
year, month, day (14)	1521.16	1714.82	0.0030	3184
year, dow, woy (45)	1505	1675.18	0.0486	2481
epoch (90)	1547.78	1711.59	0.0068	2621

⁷the actual count was 4455

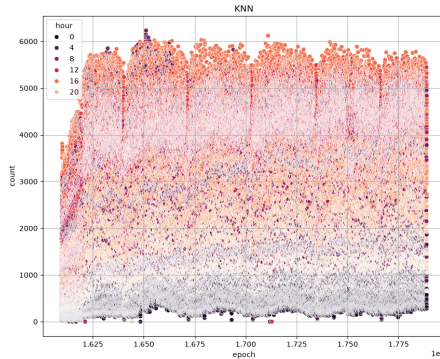


Figure 5: Regression Line using the “epoch” feature

5.3 Support Vector Machine

Feature	MAE	RMSE	Epln Var	Value
all features	1573.44	1716.7879	0.0019	2565
year, month, day	1474.65	1718.42	0.0001	2544
year, dow, woy	1574.62	1718.38	0.0001	2544
epoch	1549.77	1694.32	0.0283	2694

5.4 Decision Tree

The decision tree algorithm seem to provide the best predictions when using all the features. the below graph shows the decision tree with a dept of 10. The predictions were made with a dept of 100. Reducing the number of features had a negative impact on the prediction. Which is intuitive as there are less decision nodes, so there are fewer outcomes.

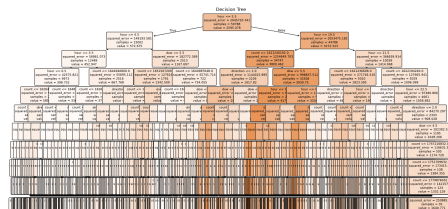


Figure 6: Decision Tree

Even predicting for other times the prediction was close, within a few hundred. Which is significantly better performance than the other algorithms

Feature	Value
all features	4656
year, month, day	2905
year, dow, woy	2955
epoch	2945

5.5 Time Series

While trying to run this algorithm, the computer running it would freeze and on occasion the app would throw an error advising that the available RAM was insufficient to complete the run. As a result, evaluation of this algorithm was not investigated.

6 Conclusion

The complexity of this project was underestimated and the complications experienced from the outset, had an impact on the final product.

Trying to gather the dataset was not a straight forward and the workaround took significant effort and additional time. This then reduced the amount of time that was available for the machine learning. This meant that a full understanding on how to evaluate the performance of a model was not achieved. Also, the resources needed to run the models were higher than expected. The computer used wasn't powerful enough, and using the likes of Amazon, Google or Azure services were cost prohibitive. Also the decision to not use AI for code generation may have been a bit naive. It could have helped speed up the coding element and left more time to research evaluating

The positives that can be taken are a new skill in the use of Selenium has been gained. This could open an interest in website testing. Also a new skill in the use of LaTeX for the preparation of the report. It made the laying out of the document easier than using the likes of word or google docs. the formatting stayed the same.

6.1 Future Development

The development of a user interface would have been a nice inclusion. However, the resources and infrastructure needed was not viable for a number of reasons.

Further exploration of the evaluation methods. This would provide solid understanding of the models, and maybe even suggest the suitability, or not, of machine learning in this application.

7 References/Bibliography

References

- [1] C. McMorrow, “‘We’ve done all we can do to fix M50,’ warns TII,” Dec. 2025. [Online]. Available: <https://www.rte.ie/news/primetime/2025/1202/1546829-m50-commuting/>
- [2] “Traffic Management Software PTV Optima.” [Online]. Available: <https://www.ptvgroup.com/en/products/traffic-management-software-ptv-optima>
- [3] “Transport Infrastructure Ireland.” [Online]. Available: <https://www.tii.ie/en/roads-tolling/operations-and-maintenance/traffic-count-data/>
- [4] “RoadPod VT | Vehicle Pneumatic Road Tube Traffic Counter.” [Online]. Available: https://www.metrocount.com/traffic-counters-classifiers/roadpod-vt?_gl=1*n02qv7*_up*MQ..*_ga*MTgzMjYyODQyMC4xNzkwMzM1Nzkw*_ga_GFY9CMWM8G*cze3OTAzMzU3OTAKbzEkZzAkDDE3OTAzMzU3OTAKajYwJGwwJGgxOTA2
- [5] “Selenium.” [Online]. Available: <https://www.selenium.dev/>
- [6] “beautifulsoup4,” Jun. 2026. [Online]. Available: <https://pypi.org/project/beautifulsoup4/>
- [7] “pandas - Python Data Analysis Library.” [Online]. Available: <https://pandas.pydata.org/>
- [8] “NumPy documentation — NumPy v2.5 Manual.” [Online]. Available: <https://numpy.org/doc/stable/>
- [9] “Getting Started.” [Online]. Available: <https://scikit-learn/stable/getting-started.html>
- [10] “Linear Regression in Machine learning,” section: Machine Learning. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>

8 Github Repository

GitHub Repository:

https://github.com/seanjcarley/DBS_FYP_2026.git